

Listák Pythonban: a list és a range()

Jegyzet és feladatlap

Hogyan dolgozz ezzel a jegyzettel?

Ezt a jegyzetet önállóan, a saját tempódban dolgozd fel: olvass, próbáld ki, aztán oldd meg a feladatokat. Háromféle rész váltja egymást:

- **Próbáld ki** – gépeld be a kódot, és futtasd. Gépeld, ne másold: a kezded is tanul.
- **Tippelj** – előbb írd le a füzetedbe, szerinted mit ír ki a program, és csak utána futtasd. Ha tévedtél, annál jobb: most tanultál valamit.
- **Feladat** – ezt már te írod meg. A feladatok kis lépésekben nehezednek, és egymásra épülnek. A nehézséget csillagok jelzik: ★ = alap, ★★ = közepes, ★★★ = nehezebb.

Néhány tudnivaló, mielőtt belevágsz:

- Amit a társaid C++-ban vektornak hívnak, az Pythonban a **lista**. A feladatok ugyanazok, úgyhogy bátran beszéljétek meg őket.
- Pythonban a behúzás számít: a ciklus és az **if** belseje négy szóközzel beljebb kezdődik.
- Ha elakadsz: olvasd el a hibaüzenetet (az utolsó sora a legfontosabb), nézd vissza a mintát, aztán kérdezz bátran.
- Nem baj, ha nem érsz a végére. Az számít, hogy amit megírtál, azt érted.

1. Miért kell a lista?

A lista sok adatot tárol egyetlen név alatt. Hogy miért jó ez, azt úgy érted meg a legjobban, ha előbb megpróbálsz nélküle.

Próbáld ki

Ebben a programban öt jegyedet tároljuk, öt külön változóban:

```
jegy1 = 5
jegy2 = 3
jegy3 = 4
jegy4 = 5
jegy5 = 2

print(jegy1, jegy2, jegy3, jegy4, jegy5)
```

Feladatok

1.1. feladat ★☆☆ - Öt változó

Gépeld be a programot, és futtasd.

1.2. feladat ★☆☆ - Még két jegy

Kaptál még két jegyet: egy 4-est és egy 5-öst. Bővítsd a programot úgy, hogy mind a hét jegyet kiírja. Számold meg: hány helyen kellett belenyúlnod a kódba?

1.3. feladat ★☆☆ - Harminc jegy

Ezt csak gondold végig, nem kell megírni. Év végére 30 jegyed lesz. Hogyan nézne ki a program? És ha előre nem is tudod, hány jegyed lesz?

Mit vettél észre?

Külön változókkal minden új adat új sort és új nevet kér. A számítógép pedig nem tudja, hogy a `jegy1` és a `jegy2` összetartozik: nem mondhatod neki, hogy „menj végig az összes jegyen”.

A lista ezt oldja meg. Képzeld el úgy, mint egy sor számozott fiókot: a sornak egy neve van, a fiókoknak pedig sorszámuk.

Sorszám (index)	0	1	2	3	4
Érték	5	3	4	5	2

Figyeld meg a felső sort: a számozás nem 1-gyel kezdődik. Erre mindjárt visszatérünk.

2. Az első listánk

Egy listát egyetlen sorral hozol létre, az elemeit pedig a sorszámukkal éred el.

Próbáld ki

```
jegyek = [5, 3, 4, 5, 2]

print(jegyek[0])
```

Nézzük meg darabonként:

- `jegyek` – a lista neve. Ezt te választod, mint bármely változónál.
- `[5, 3, 4, 5, 2]` – az elemek, szögletes zárójelben, vesszővel elválasztva.
- `jegyek[0]` – a lista egyik eleme. A szögletes zárójelben álló szám az **index**.

Tippelj

Mit ír ki a fenti program? Írd le a tipped, aztán futtasd.

Most írd át a `jegyek[0]`-t `jegyek[1]`-re, majd `jegyek[4]`-re. Mindkétszer tippelj előbb.

Válaszolj a füzetedben:

- Mi az első elem indexe?
- Mi az utolsó elem indexe, ha öt elem van?
- Mi lenne az utolsó elem indexe, ha száz elem lenne?

Kísérlet – az egész lista egyszerre

Írd a program végére ezt a sort:

```
print(jegyek)
```

Mit látsz?

Tipp: Ha nem érted, mit csinál a programod, írasd ki így az egész listát. Hibakeresésnél ez a leggyorsabb segítség.

Kísérlet – mi van a lista végén túl?

Írasd ki a `jegyek[5]` értékét. Olvasd el a hibaüzenet utolsó sorát. Mit mond a Python?

Most próbáld ki a `jegyek[-1]` és a `jegyek[-2]` értékét. Ezek nem adnak hibát. Fogalmazd meg a füzetedben, mit jelent a negatív index.

Feladatok

2.1. feladat ★☆☆ - Lépésszámláló

Hozz létre egy `lepesek` nevű listát négy nap lépésszámával: 8200, 10450, 6300, 12000. Írd ki az első és az utolsó elemét.

2.2. feladat ★☆☆ - Egy sorban

Írd ki mind a négy elemet egy sorba, de egyesével, az indexükkel hivatkozva rájuk. (Ciklus most még nem kell.)

2.3. feladat ★☆☆ - Két nap összege

Írd ki az első két nap lépésszámának összegét. A program számoljon, ne te.

2.4. feladat ★☆☆ - Tizedes törtek

A listában nem csak egész számok lehetnek. Próbáld ki ezt a sort, és írd ki a középső elemet:

```
arak = [299.9, 1250.5, 89.0]
```

Kellett bármit másképp csinálnod, mint az egész számoknál?

2.5. feladat ★☆☆ - Kedvenc számok

Hozz létre egy saját listát a kedvenc öt számodból, és írd ki őket fordított sorrendben.

3. Elemek módosítása

A lista egy eleme pontosan úgy viselkedik, mint egy sima változó: kaphat új értéket, lehet vele számolni, be lehet olvasni.

Próbáld ki

Az utolsó jegyedet, a 2-est kijavítottad 4-esre:

```
jegyek = [5, 3, 4, 5, 2]

print("Elotte:", jegyek[4])
jegyek[4] = 4
print("Utana:", jegyek[4])
```

Tippelj

Mit ír ki ez a részlet? Kövesd sorról sorra papíron, és csak utána futtasd.

```
v = [10, 20, 30]

v[0] = v[1] + v[2]
v[1] = v[1] * 2
v[2] = v[2] + 1

print(v[0], v[1], v[2])
```

Feladatok

A feladatokhoz használd újra a `lepesek` listát: 8200, 10450, 6300, 12000.

3.1. feladat ★☆☆ - Javítás

Kiderült, hogy a második napon valójában 11000 lépést tettél meg. Javítsd ki a listában, és írd ki a javított értéket. Vigyázz: mi a második nap indexe?

3.2. feladat ★☆☆ - Esti séta

Az első napon este még sétáltál 500 lépést. Növeld meg az első elemet 500-zal. Ne a végeredményt írd be, a program számolja ki.

3.3. feladat ★☆☆ - Beolvasott adat

A negyedik nap adatát olvasd be a billentyűzetről. Egy listaelembe ugyanúgy lehet beolvasni, mint egy változóba:

```
lepesek[3] = int(input("Negyedik nap: "))
```

Írasd ki utána az egész listát, hogy lásd a változást.

3.4. feladat ★☆☆ - Csere

Cseréld meg az első és az utolsó elemet. Először próbáld ki ezt a két sort, és írasd ki az eredményt:

```
lepesek[0] = lepesek[3]
lepesek[3] = lepesek[0]
```

Mi romlott el, és miért? Javítsd ki úgy, hogy a csere tényleg működjön. **Tipp:** kell egy harmadik, segédváltozó.

3.5. feladat ★☆☆ - Csere egy sorban

Pythonban a cserére van egysoros megoldás is. Próbáld ki, és írasd ki utána a listát:

```
lepesek[0], lepesek[3] = lepesek[3], lepesek[0]
```

Hasonlítsd össze a 3.4. megoldásoddal. Ugyanazt csinálja?

4. A lista hossza, a range() és a bejárás

A lista igazi ereje az, hogy egy **for** ciklus végig tud menni az összes elemén, akárhány is van belőle. Ehhez két segítődd lesz: a **len()** és a **range()**.

Próbáld ki - hány elem van?

A lista hosszát a **len()** árulja el:

```
jegyek = [5, 3, 4, 5, 2]

print("Jegyek száma:", len(jegyek))
```

Adj hozzá a szögletes zárójelbe még két jegyet, és futtasd újra.

Tippelj

Mit ír ki ez a sor? És miért pont azt?

```
print(jegyek[len(jegyek) - 1])
```

Próbáld ki - az index lehet változó is

Eddig mindig számot írtál a szögletes zárójelbe. Változót is lehet:

```
i = 2
print(jegyek[i])
```

Írd át az **i** értékét 0-ra, majd 3-ra. Mindig az **i**-edik elemet kapod.

Ha lenne valami, ami az `i`-t sorban 0-tól 4-ig lépteti, az egész listát ki tudnánk írni. Pontosan ezt csinálja a `range()`.

A `range()` egy számmal

A `range()` számsorozatot ad a `for` ciklusnak. A ciklusváltozó sorban felveszi a sorozat tagjait.

```
for i in range(5):  
    print(i)
```

Mielőtt futtatod, tippelj: melyik az első kiírt szám? Melyik az utolsó? Kiírja az 5-öt?

Futtasd, és számold meg, hány számot írt ki. Aztán próbáld ki `range(3)`-mal és `range(1)`-gyel is.

Amit megfigyelhettél: a `range(5)` 0-tól indul, és az 5 **előtt** megáll. Ez éppen öt szám, és éppen egy ötelemű lista indexei.

A `range()` két számmal

Megadhatod azt is, honnan induljon a sorozat:

```
for i in range(2, 6):  
    print(i)
```

Tippelj, aztán futtasd. Az első szám (a kezdet) benne van a sorozatban, a második (a vég) most sincs.

A `range()` három számmal

A harmadik szám a lépésköz: ennyivel nő a ciklusváltozó minden körben.

```
for i in range(0, 10, 2):  
    print(i)
```

A lépésköz negatív is lehet. Ilyenkor a sorozat visszafelé halad:

```
for i in range(5, 0, -1):  
    print(i)
```

Tippelj mindkettőnél, aztán futtasd.

Figyelem! A `range()` végértéke soha nincs benne a sorozatban – akkor sem, ha visszafelé haladsz.

Tippelj - totó a `range()`-ról

Töltsd ki a középső oszlopot fejből, gép nélkül.

Kifejezés	A tippem	A Python szerint
<code>range(6)</code>		
<code>range(3, 7)</code>		
<code>range(2, 11, 3)</code>		
<code>range(10, 0, -2)</code>		
<code>range(4, -1, -1)</code>		
<code>range(5, 2)</code>		

Utána ellenőrizd géppel, és írd be a jobb oldalra, amit a Python mond. Az ellenőrzéshez nem kell ciklus, ez a sor egyszerre megmutatja a számokat:

```
print(list(range(6)))
```

Az utolsó sor meglepő lehet. Magyarázd meg egy mondatban, miért azt kaptad.

Próbáld ki - a bejárás

Most rakjuk össze a kettőt: a `len()` megmondja, hány elem van, a `range()` pedig előállítja az indexeket.

```
jegyek = [5, 3, 4, 5, 2]

for i in range(len(jegyek)):
    print(jegyek[i], end=" ")
print()
```

A `range(len(jegyek))` itt ugyanaz, mint a `range(5)`: az `i` sorban a 0, 1, 2, 3, 4 értékeket veszi fel. Így a `jegyek[i]` sorban az összes elemet jelenti.

Az `end=" "` azt mondja a `print`-nek, hogy sortörés helyett szóközt tegyen a kiírás végére. Az üres `print()` a végén lezárja a sort.

Kísérletek

- Adj a listához még három jegyet. Hozzá kellett nyúlnod a ciklushoz? Emlékezz vissza az 1.2. feladatra.
- Írd át a ciklust erre: `range(len(jegyek) + 1)`. Mi történik? Miért? (Gondolj a 2. fejezet kísérletére.)
- Írd át erre: `range(1, len(jegyek))`. Melyik elem marad ki?

Próbáld ki - bejárás index nélkül

Pythonban a `for` közvetlenül az elemeken is végig tud menni:

```
for jegy in jegyek:
    print(jegy, end=" ")
print()
```

Itt a `jegy` változó nem indexeket kap, hanem sorban magukat az elemeket. Ez rövidebb, de nem tudod, hányadik elemnél jársz.

Melyiket mikor használd?

- Ha csak az értékek kellenek, elég a `for jegy in jegyek`.
- Ha a sorszám is kell, vagy módosítani akarsz az elemeket, akkor `for i in range(len(jegyek))`.

Feladatok

Használd ezt a listát, most már egy egész hét adataival:

```
lepesek = [8200, 10450, 6300, 12000, 9100, 15300, 4800]
```

4.1. feladat ★☆☆ - Egész hét

Írd ki az összes elemet, mindegyiket új sorba. Oldd meg mindkét bejárással.

4.2. feladat ★☆☆ - Sorszámozva

Írd ki a nap sorszámát is, 1-től számozva:

```
Nap 1 - 8200 lepes
Nap 2 - 10450 lepes
...
```

Tipp: az index 0-tól indul, a sorszám 1-től. Mennyi a különbség? Melyik bejárás kell ide?

4.3. feladat ★☆☆ - Visszafelé

Írd ki az elemeket visszafelé, az utolsótól az elsőig. **Tipp:** a `range()` három számmal. Honnan indul az `i`, mi előtt áll meg, és mennyi a lépésköz?

4.4. feladat ★☆☆ - Minden második

Írd ki csak minden második napot: az első, a harmadikat, az ötödiket és a hetediket. Kétféle megoldás is van: az egyik a `range()` lépésközét használja, a másik egy `if`-et. Keresd meg mindkettőt.

4.5. feladat ★☆☆ - Tízezres napok

Írd ki csak azokat a napokat, amikor legalább 10000 lépést tettél meg. **Tipp:** a ciklus belsejébe `if` kerül.

4.6. feladat ★☆☆ - Bónusz lépések

Mindegyik napra jóváírnak 1000 bónusz lépést. Először próbáld ki ezt, és írasd ki utána a listát:

```
for x in lepesek:
    x = x + 1000
```

Megváltozott a lista? Most oldd meg indexekkel, a `range(len(lepesek))` segítségével. Mi a különbség a két változat között?

5. Bővítés és beolvasás

A lista futás közben is nőhet: az `append` új elemet tesz a végére. Így azt is el tudod tárolni, amit a felhasználó gépel be.

Tippelj

Ebben a részletben három kiírás van. Mindhárom elé írd oda a tippet, és csak utána futtasd.

```
szamok = []
print(len(szamok))

szamok.append(7)
szamok.append(3)
print(len(szamok))

print(szamok[0], szamok[1])
```

Amit megfigyelhettél:

- Az üres szögletes zárójel üres listát jelent, a hossza 0.
- Az `append(x)` az `x` értéket a lista **végére** teszi, és a hossz eggyel nő.

Próbáld ki - beolvasás a billentyűzetről

```
szamok = []

db = int(input("Hany szamot adsz meg? "))

for i in range(db):
    x = int(input())
    szamok.append(x)

print("Ezeket adtad meg:", end=" ")
for i in range(len(szamok)):
    print(szamok[i], end=" ")
print()
```

A program két ciklusból áll. Az első beolvas és eltárol, a második kiír. A két ciklus között az adatok a listában várnak.

Futtasd többször, más-más darabszámmal. Próbáld ki 0-val is.

Tipp: Minden számot külön sorba írd, és mindegyik után nyomj Entert.

Figyeld meg az első ciklust: a `range(db)` itt nem indexeket ad, hanem egyszerűen megszámlolja, hányszor fusson le a ciklus.

Kísérlet - sok egyforma elem

Íradd ki ezt a listát:

```
nullak = [0] * 5
```

Mit látsz? Most hasonlítsd össze ezt a kettőt: írad ki mindkettő hosszát és elemeit.

```
a = [0] * 3
b = [3]
```

Fogalmazd meg a füzetedben egy mondatban, mi a különbség a kettő között.

Feladatok

5.1. feladat ★☆☆ - Három szám

Hozz létre egy üres listát. Tedd bele `append`-del az 1, 2, 3 számokat, majd írd ki ciklussal.

5.2. feladat ★☆☆ - Egytől tízig

Töltsd fel a listát 1-től 10-ig a számokkal. Most már ne tíz `append` sort írd, hanem egy ciklust. **Tipp:** melyik `range()` adja éppen az 1, 2, ..., 10 számokat?

5.3. feladat ★☆☆ - Négyzetszámok

Töltsd fel a listát az első tíz négyzetszámmal: 1, 4, 9, 16, ... Írd ki az elemeket.

5.4. feladat ★☆☆ - Fordított hőmérsékletek

Kérj be öt hőmérsékletet, tárold el őket, majd írd ki fordított sorrendben. (A 4.3. feladat segít.)

5.5. feladat ★☆☆ - Nulláig

Kérj be számokat addig, amíg a felhasználó 0-t nem ír be. A 0 már ne kerüljön a listába. A végén írd ki, hány számot adott meg, és sorold fel őket. **Tipp:** itt nem tudod előre a darabszámot, ezért `while` ciklus kell.

5.6. feladat ★☆☆ - Kell-e tárolni?

Gondold végig, és írd le egy mondatban: az 5.4. feladatot meg lehetett volna oldani lista nélkül, egyetlen változóval?

6. Összegzés, átlag, megszámlálás

Ha egy ciklus végigmegy a listán, közben gyűjtögethet is: összeadhatja az elemeket, vagy megszámlálhatja, hány felel meg egy feltételnek.

Próbáld ki - összegzés

```
jegyek = [5, 3, 4, 5, 2]

osszeg = 0
for i in range(len(jegyek)):
    osszeg = osszeg + jegyek[i]

print("Összeg:", osszeg)
```

Az **osszeg** olyan, mint egy persely: üresen indul, és minden körben beledobjuk a következő elemet.

Kövesd végig papíron, mi történik. Az első két sort kitöltöttem, a többi te írd be:

i értéke	jegyek[i]	osszeg a kör végén
0	5	5
1	3	8
2		
3		
4		

Egyezik az utolsó sor azzal, amit a program kiírt?

Kísérlet

Tedd át az **osszeg = 0** sort a ciklus belsejébe, a **for** sora alá, behúzva. Mit ír ki most a program? Miért éppen azt?

Tippelj - átlag

Az átlag az összeg osztva a darabszámmal. A darabszámot ismered: **len(jegyek)**. Írd az összegző program végére ezt a két sort:

```
print("Átlag:", osszeg / len(jegyek))
print("Átlag:", osszeg // len(jegyek))
```

Számold ki fejben, mennyi 19 osztva 5-tel. Aztán futtasd. Melyik sor adta azt, amit vártál?

Figyelem! A **/** tizedes törtet ad. A **//** egészszosztás: levágja a tizedeseket. Átlaghoz mindig az egyszerű **/** jel kell.

Próbáld ki - megszámlálás

Hány ötösöd van?

```
jegyek = [5, 3, 4, 5, 2]

db = 0
for i in range(len(jegyek)):
    if jegyek[i] == 5:
        db = db + 1

print("Ötosok száma:", db)
```

Hasonlítsd össze az összegzéssel. Mi egyezik a két programban, és mi a két különbség?

Tipp: Az összegre van kész függvény is: `sum(jegyek)`. Használd ellenőrzésre. A feladatokban viszont te írd meg a ciklust: amint feltétel is kerül az összegzésbe vagy a számlálásba, úgyis arra lesz szükséged.

Feladatok

Az első három feladathoz a heti lépésszámokat használd:

```
lepesek = [8200, 10450, 6300, 12000, 9100, 15300, 4800]
```

6.1. feladat ★☆☆ - Heti összeg

Hány lépést tettél meg a héten összesen?

6.2. feladat ★☆☆ - Napi átlag

Mennyi a napi átlag? Tizedesjegyekkel együtt írd ki.

6.3. feladat ★☆☆ - Hány tízezres nap?

Hány napon érted el a 10000 lépést?

A következő kettőhöz egy téli hét hőmérsékletei kellenek:

```
hom = [-3, 2, 0, -5, 4, 1, -1]
```

6.4. feladat ★☆☆ - Fagyos napok

Hány napon volt fagy, vagyis 0 foknál hidegebb?

6.5. feladat ★★☆☆ - Fagypont felett

Mennyi a fagypont feletti értékek összege? Itt az összegzést és a feltételt kell összeraknod.

6.6. feladat ★★☆☆ - Átlag feletti jegyek

Kérj be hat jegyet a billentyűzetről (5. fejezet). Írd ki az átlagukat, majd azt, hogy hány jegy jobb az átlagnál.

Gondold végig: hányszor kellett végigmenned az adatokon? Meg tudtad volna oldani úgy, hogy a jegyeket nem tárolod el?

7. Legnagyobb és legkisebb elem

A legnagyobb elemet úgy találod meg, hogy megjegyzed az eddigi legnagyobbat, és minden új elemnél megnézed, nagyobb-e nála.

Gondold végig gép nélkül

Valaki egyesével mutat neked számkártyákat, és mindegyiket csak egy pillanatra látod. A végén meg kell mondanod a legnagyobbat. Mit csinálsz közben fejben? Írd le két mondatban, mielőtt továbbolvasol.

Próbáld ki

```
pontok = [42, 67, 55, 91, 38]

legnagyobb = pontok[0]
for i in range(1, len(pontok)):
    if pontok[i] > legnagyobb:
        legnagyobb = pontok[i]

print("Legnagyobb:", legnagyobb)
```

Kövesd végig papíron: melyik körökben változik a `legnagyobb` értéke, és melyikekben nem?

Nézd meg a `range()`-et: most két számot kapott. Miért indulhat az `i` 1-ről, és nem 0-ról?

Kísérlet - miért nem nulláról indulunk?

Kézenfekvőnek tűnik így kezdeni: `legnagyobb = 0`. Írd át a programot erre, és próbáld ki ezzel a listával:

```
pontok = [-5, -2, -9]
```

Mit ír ki? Miért rossz ez az eredmény?

Figyelem! A legnagyobb elem keresését ne 0-ról indítsd! Mindig a lista első eleméből indulj ki.

Tipp: Erre is van kész függvény: `max(pontok)` és `min(pontok)`. Ellenőrzésre jók, de azt nem mondják meg, **hol** van a legnagyobb elem. Ahhoz a saját ciklusod kell.

Feladatok

7.1. feladat ★☆☆ - Legkisebb

Írd át a mintaprogramot úgy, hogy a legkisebb pontszámot keresse meg. Hány helyen kellett érdemben változtatnod?

7.2. feladat ★☆☆ - Terjedelem

Írd ki egy programban a legnagyobb és a legkisebb pontszámot, valamint a kettő különbségét.

7.3. feladat ★☆☆ - Hol a legnagyobb?

Hányadik indexen van a legnagyobb elem? **Tipp:** most ne az értéket jegyezd meg, hanem a helyét.

```
max_index = 0
```

A ciklusban a `pontok[i]`-t a `pontok[max_index]`-szel hasonlítsd össze.

7.4. feladat ★☆☆ - Leghidegebb nap

Használd a téli hét hőmérsékleteit: -3, 2, 0, -5, 4, 1, -1. Írd ki, hányadik napon volt a leghidegebb (1-től számozva), és hány fok volt aznap.

7.5. feladat ★☆☆ - Két egyforma

Futtasd a 7.3. megoldásodat ezzel a listával: 3, 9, 4, 9, 1. Két egyforma legnagyobb van benne. Melyiknek az indexét kaptad? Mi történik, ha a > jelet >= jelre cseréled? Magyarázd meg.

8. Keresés: van-e benne, és hol?

Azt, hogy egy érték szerepel-e a listában, egy logikai változó jegyzi meg: hamisról indul, és igazra vált, ha megtaláltuk.

Próbáld ki

```
szamok = [4, 8, 15, 16, 23, 42]

keresett = int(input("Melyik számot keresed? "))

van = False
for i in range(len(szamok)):
    if szamok[i] == keresett:
        van = True

if van:
    print("Benne van.")
else:
    print("Nincs benne.")
```

Futtasd háromszor: keresd a 15-öt, a 42-t és a 7-et.

Figyeld meg a szerkezetet. A ciklus csak keres és megjegyez. A válasz kiírása a ciklus **után** történik, amikor már minden elemet láttunk. Ezt a behúzás mutatja: az utolsó **if** már nincs beljebb húzva.

Kísérlet - a leggyakoribb hiba

Sokan így írják meg elsőre. Próbáld ki, és keresd vele a 15-öt:

```
for i in range(len(szamok)):
    if szamok[i] == keresett:
        print("Benne van.")
    else:
        print("Nincs benne.")
```

Mit ír ki? Miért? Fogalmazd meg egy mondatban, mikor mondhatjuk ki biztosan, hogy egy szám nincs a listában.

Tipp: A Python egy szóval is megmondja, szerepel-e egy érték a listában: a `keresett in szamok` kifejezés értéke `True` vagy `False`. Ellenőrzésre használd; azt nem árulja el, hol van az elem.

Feladatok

8.1. feladat ★☆☆ - Hol van?

Egészítsd ki a mintaprogramot: ha megvan a keresett szám, azt is írja ki, hányadik indexen találta.

Tipp: vegyél fel egy `hely = -1` változót. Miért jó kezdőérték a -1?

8.2. feladat ★☆☆ - Van-e egyes?

A jegyeid: 5, 3, 4, 5, 2, 4, 3. Van köztük egyes? Írj rá programot. Aztán írd át egy jegyet 1-esre, és ellenőrizd, hogy a program észreveszi.

8.3. feladat ★☆☆ - Mindegyik?

Igaz-e, hogy **minden** jegyed legalább hármas? **Tipp:** fordítva gondolkodj, mint eddig. Induljon a változó igazról (`mind = True`), és akkor váltson hamisra, ha találsz egy kivételt.

8.4. feladat ★★☆☆ - A hatodik szám

Kérj be öt számot egy listába, majd kérj be egy hatodikat. Írd ki, hogy a hatodik szerepelt-e az első öt között.

8.5. feladat ★★☆☆ - Megállás az első találatnál

A mintaprogram akkor is végigmegy az egész listán, ha már az első elem a keresett. Ez a `while` ciklus megáll az első találatnál:

```
i = 0
while i < len(szamok) and szamok[i] != keresett:
    i = i + 1
```

Futtasd, és írasd ki utána az `i` értékét, többféle keresett számmal. Mit jelent, ha az `i` kisebb, mint a lista hossza? És ha éppen egyenlő vele? Fejezd be a programot úgy, hogy kiírja a választ.

9. Kihívások

Ezekhez a feladatokhoz nincs új tudnivaló: mindent ismersz hozzá, csak össze kell raknod. Nem kell sorban haladnod, válaszd azt, amelyik megtetszik.

9.1. feladat ★★☆☆ - Csillagdiagram

Adott a lista: 3, 5, 1, 4. Rajzolj minden elemhez egy sort, amelyben annyi csillag van, amennyi az elem értéke:

```
***
*****
*
*****
```

Tipp: ciklus a ciklusban, ahogy az alakzatok rajzolásánál csináltad. Egy csillagot sortörés nélkül így írsz ki: `print(" ", end="")`.

9.2. feladat ★★☆☆ - Szétválogatás

Adott a lista: 12, 7, 4, 9, 20, 15, 8. Hozz létre két üres listát `parosak` és `paratlanok` néven. Válogasd szét beléjük a számokat, majd írd ki mindkettőt.

9.3. feladat ★★☆☆ - Fordított másolat

Készíts egy új listát, amelyben az eredeti elemei fordított sorrendben szerepelnek. Az eredeti lista maradjon változatlan. Írd ki mindkettőt.

9.4. feladat ★★☆☆ - Növekvő-e?

Döntsd el, hogy a lista elemei növekvő sorrendben vannak-e. Ehhez a szomszédokat kell összehasonlítnod: a `v[i]` elemet a `v[i + 1]` elemmel. Vigyázz: milyen `range()` kell, hogy a `v[i + 1]` még létezzen? Próbáld ki mindkét listával:

```
a = [1, 3, 7, 10]
b = [1, 5, 4, 8]
```

9.5. feladat ★★☆☆ - Legnagyobb melegedés

A téli hét hőmérsékletei: -3, 2, 0, -5, 4, 1, -1. Melyik két egymást követő nap között nőtt a legtöbbet a hőmérséklet, és hány fokkal? Előbb számold ki fejben, hogy ellenőrizni tudd a programodat.

9.6. feladat ★★★ - Dolgozatstatisztika

Kérd be egy osztály dolgozatainak pontszámait (0 és 100 között). Előbb azt kérdezd meg, hány dolgozat van. Írd ki:

- az átlagpontszámot,
- a legjobb és a leggyengébb eredményt,
- hány dolgozat lett jobb az átlagnál,
- volt-e 100 pontos dolgozat.

Ez a feladat szinte az egész jegyzetet összefoglalja. Építsd lépésenként: egy rész megírása után futtasd, és csak utána írd a következőt.

9.7. feladat ★★★ - Megfordítás helyben

Fordítsd meg a lista elemeinek sorrendjét új lista nélkül. **Tipp:** a 3.4. feladat cseréjét használd. Cseréld meg az elsőt az utolsóval, a másodikat az utolsó előttivel, és így tovább.

Először menj végig a cserékkel az egész listán, és nézd meg az eredményt. Mi történt? Meddig kell tehát mennie a ciklusnak?

Összefoglaló táblázat (puska)

Ennyi kell ahhoz, hogy listával dolgozz.

Művelet	Jelentés
<code>v = [5, 3, 4]</code>	Lista kezdő elemekkel
<code>v = []</code>	Üres lista
<code>v = [0] * 5</code>	Öt darab nulla
<code>v[i]</code>	Egy elem elérése
<code>v[i] = 10</code>	Egy elem módosítása
<code>len(v)</code>	Elemek száma
<code>v[0]</code>	Első elem
<code>v[len(v) - 1]</code> vagy <code>v[-1]</code>	Utolsó elem
<code>v.append(x)</code>	Új elem a végére
<code>for i in range(len(v)):</code>	Bejárás indexszel
<code>for x in v:</code>	Bejárás index nélkül
<code>x = int(input())</code>	Egész szám beolvasása

A `range()` három alakja:

Alak	Mit jelent?	Példa	A sorozat
<code>range(n)</code>	0-tól <code>n</code> előttig	<code>range(4)</code>	0, 1, 2, 3
<code>range(a, b)</code>	<code>a</code> -tól <code>b</code> előttig	<code>range(2, 5)</code>	2, 3, 4
<code>range(a, b, c)</code>	<code>a</code> -tól <code>b</code> előttig, <code>c</code> lépésközzel	<code>range(1, 8, 3)</code>	1, 4, 7
negatív lépésköz	visszafelé halad	<code>range(3, -1, -1)</code>	3, 2, 1, 0

Figyelem! Két dolog, amit soha ne felejts el: az indexek 0-tól `len(v) - 1`-ig tartanak, és a `range()` végértéke soha nincs benne a sorozatban.